

PCB Classification

IT사업부 | 제품 개발2파트
김현주 프로

Index

PCB Classification

01

EDA 및 데이터 전처리

1. 데이터 설명
2. EDA
3. 데이터 전처리

02

이진 분류

1. 데이터 라벨링
2. 모델 비교 및 평가
 - 2.1 하이퍼파라미터 설명 – 옵티마이저
3. EfficientNet-B3 모델 평가

03

다중 클래스 분류

1. 모델 비교
2. 정확도 향상 과정
 - 2.1 증강 후 모델 비교

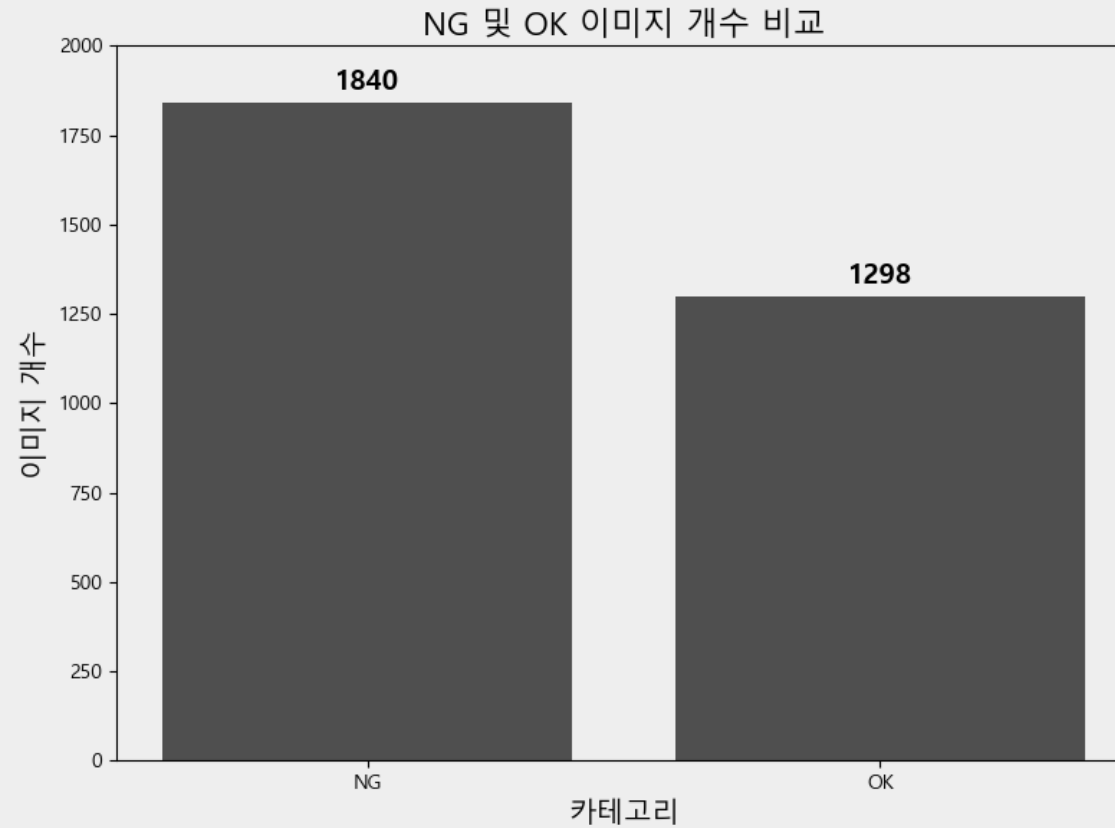
01

EDA 및 데이터 전처리

1. 데이터 설명
2. EDA
3. 데이터 전처리

01 EDA 및 데이터 전처리

1. 데이터 설명



→ NG와 OK 데이터 양에 차이가 있지만 이진 분류 학습에 크게 문제가 생길 정도는 아니라고 판단하여 데이터 균형을 맞추지 않음.

01 EDA 및 데이터 전처리

1. 데이터 설명

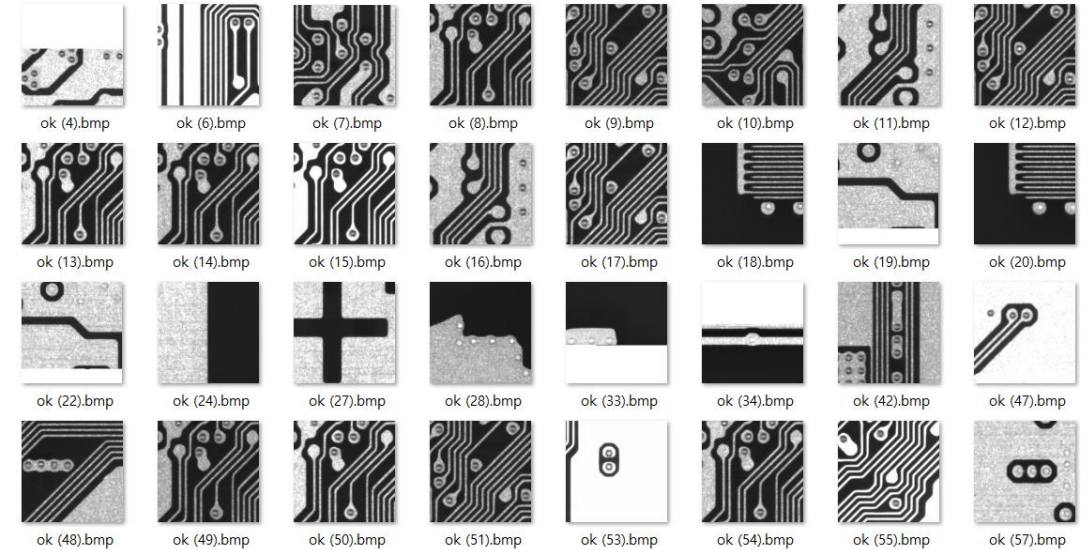
NG (PCB 결함 이미지)



NG 파일 라벨_총 18개

bichim, chui, dust, dust_Nick, eemool, Island, LVH, miboo, miboosik, milchack,
nick, Nick_Dust, nogwang, open, pinhole, pyomyun, scratch, short

OK (PCB 정상 이미지)

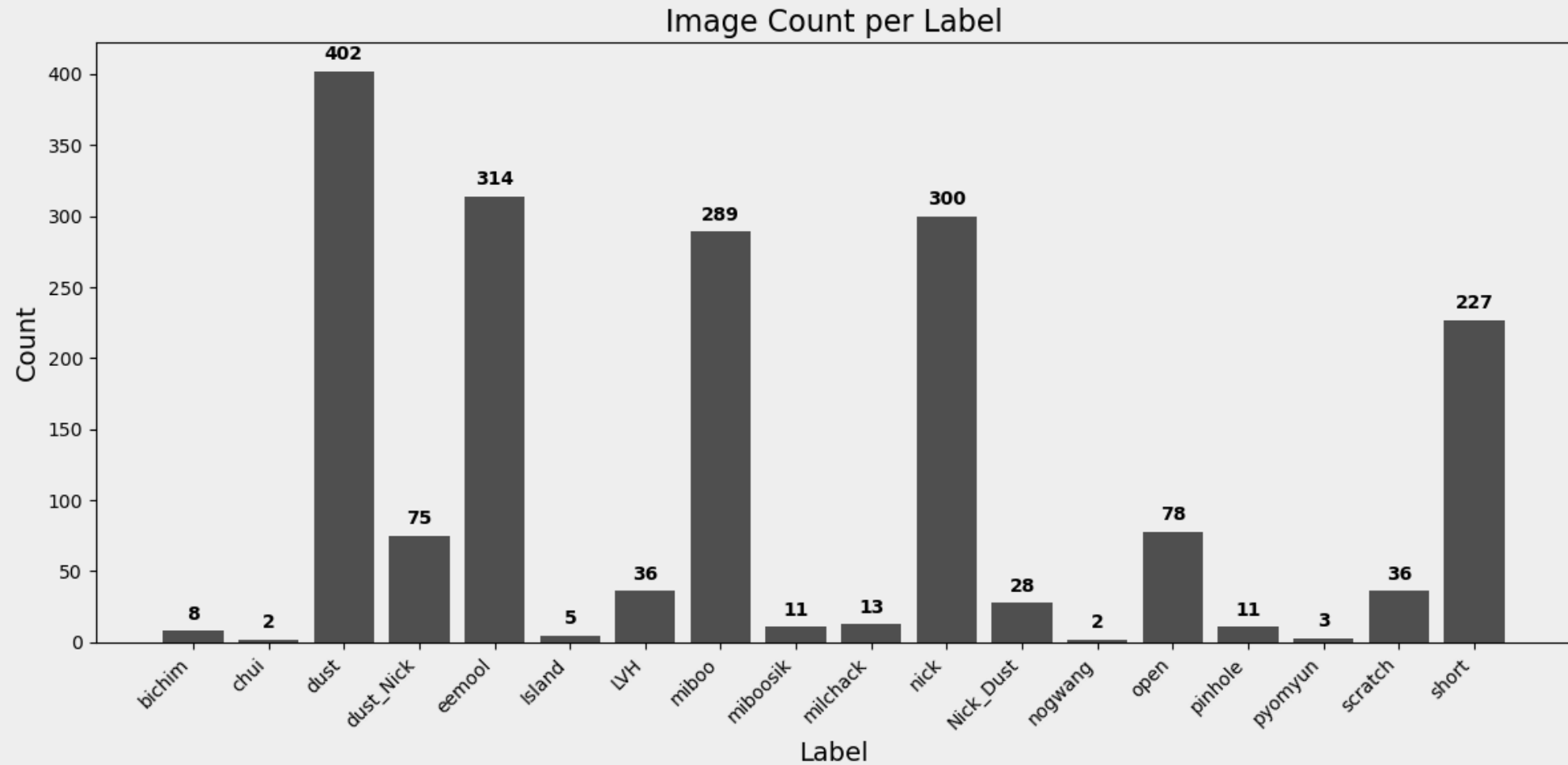


OK 파일 라벨_총 1개

OK

01 EDA 및 데이터 전처리

2. EDA



01 EDA 및 데이터 전처리

2. EDA

NG라벨별 의미 파악

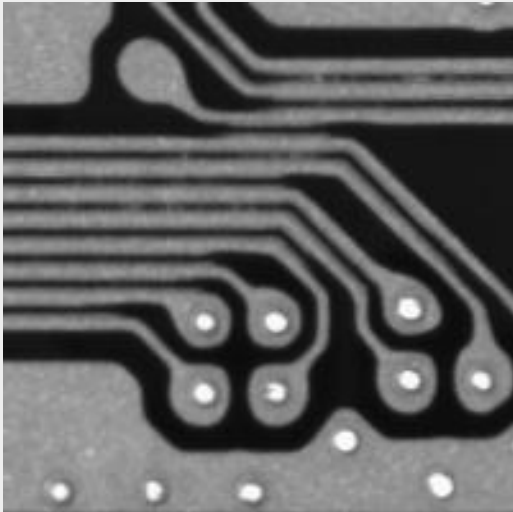
bichim	비침 (기판이 균일하지 않아 투명하게 보이는 현상)	milchack	밀착 불량 (기판과 패턴 간 밀착이 제대로 이루어지지 않음)
chui	취 (기판의 표면에 불균일한 흐름 또는 작은 결함이 발생한 현상)	nick	찍힘 (기판에 작은 찍힘이 발생한 결함)
dust	먼지 (기판 표면에 먼지가 붙은 결함)	nick_dust	찍힘과 먼지 (찍힘과 먼지가 결합된 결함)
dust_nick	먼지와 찍힘(먼지와 찍힘이 결합된 결함)	nogwang	노광 불량 (노광 공정에서 발생한 결함)
eemool	이물 (기판 표면에 이물질이 붙어 있는 상태)	open	열림 (패턴에서 도체가 끊긴 상태)
Island	섬 (기판 표면에 고립된 영역이 형성된 결함)	pinhole	핀홀 (기판에 작은 구멍이 생긴 결함)
lvh	저전압 구멍 (기판에서 저전압 문제로 생긴 구멍)	pyomyun	표면 불량 (기판의 표면에 생긴 결함)
miboo	미부 (불완전한 패턴 형성으로 인해 결함이 발생한 상태)	scratch	스크래치 (기판 표면에 생긴 결함)
miboosik	미부식 (기판 표면이 제대로 부식되지 않은 상태)	short	쇼트 (회로에서 도체 간에 불필요한 연결이 발생한 상태)

→ miboo와 miboosik, dust_nick과 nick_dust가 중복된다고 판단

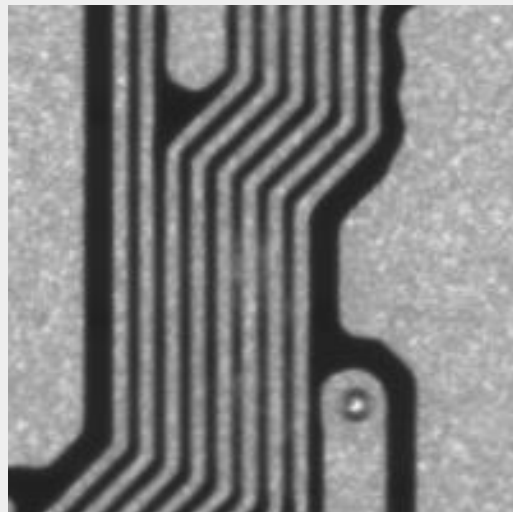
01 EDA 및 데이터 전처리

2. EDA

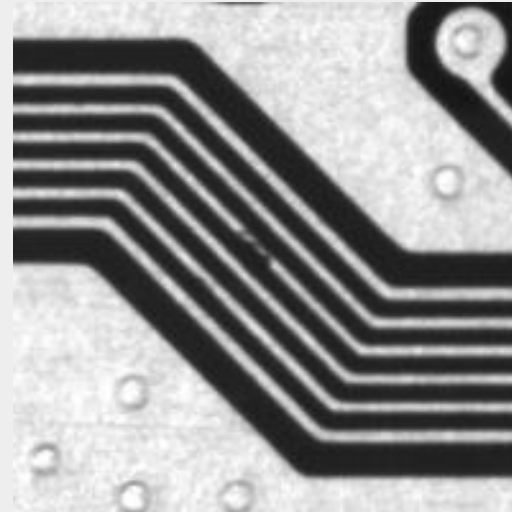
miboo



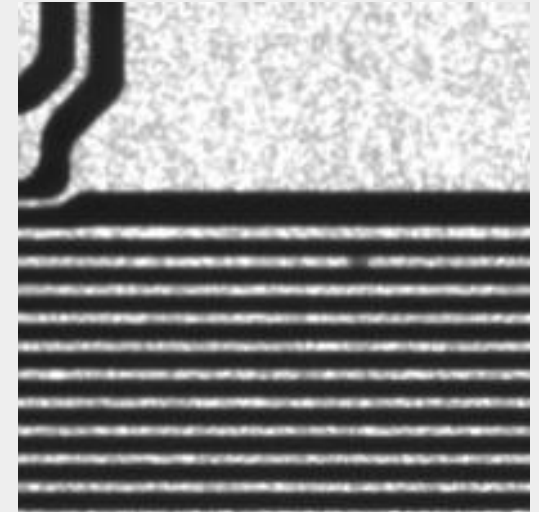
miboosik



dust_nick



nick_dust



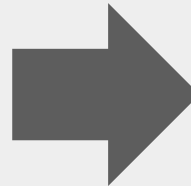
→ miboo와 miboosik, dust_nick과 nick_dust가 중복된다고 판단

01 EDA 및 데이터 전처리

2. EDA

중복라벨통합

bichim: 8개의 이미지 파일
 chui: 2개의 이미지 파일
 dust: 402개의 이미지 파일
 dust_nick: 75개의 이미지 파일
 eemool: 314개의 이미지 파일
 Island: 5개의 이미지 파일
 LVH: 36개의 이미지 파일
 miboo: 289개의 이미지 파일
 miboosik: 11개의 이미지 파일
 milchack: 13개의 이미지 파일
 nick: 300개의 이미지 파일
 Nick_Dust: 28개의 이미지 파일
 nogwang: 2개의 이미지 파일
 open: 78개의 이미지 파일
 pinhole: 11개의 이미지 파일
 pyomyun: 3개의 이미지 파일
 scratch: 36개의 이미지 파일
 short: 227개의 이미지 파일
 라벨이 없는 이미지 파일: 0개

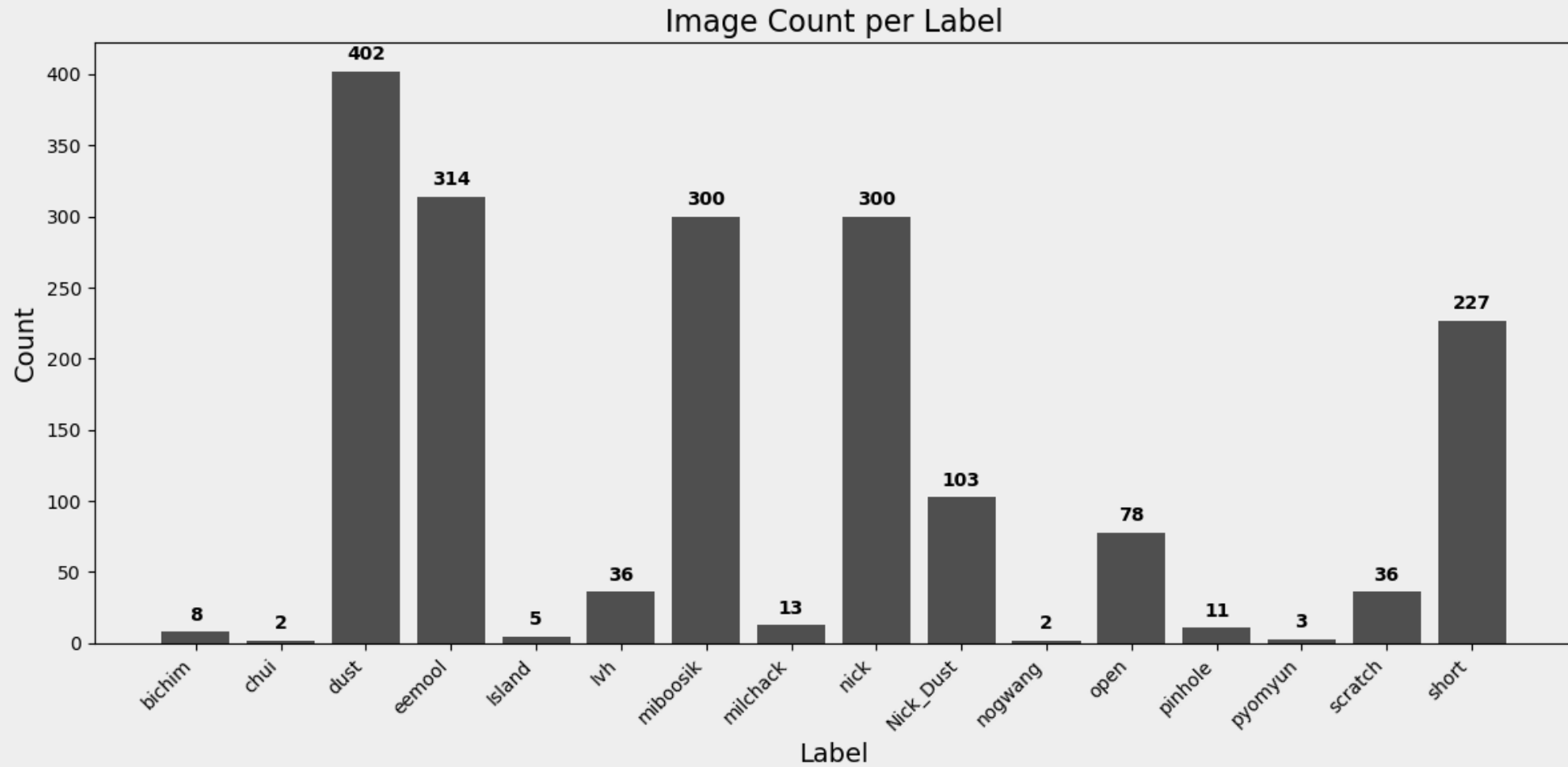


dust_nick, Nick_Dust → Nick_Dust
 miboo, miboosik → miboosik

bichim: 8개의 이미지 파일
 chui: 2개의 이미지 파일
 dust: 402개의 이미지 파일
 eemool: 314개의 이미지 파일
 Island: 5개의 이미지 파일
 lvh: 36개의 이미지 파일
 miboosik: 300개의 이미지 파일
 milchack: 13개의 이미지 파일
 nick: 300개의 이미지 파일
 Nick_Dust: 103개의 이미지 파일
 nogwang: 2개의 이미지 파일
 open: 78개의 이미지 파일
 pinhole: 11개의 이미지 파일
 pyomyun: 3개의 이미지 파일
 scratch: 36개의 이미지 파일
 short: 227개의 이미지 파일
 라벨이 없는 이미지 파일: 0개

01 EDA 및 데이터 전처리

2. EDA



01 EDA 및 데이터 전처리

3. 데이터 전처리

이미지 크기 확인 및 224 * 224 사이즈 통일

NG 이미지 크기 확인:

폴더 'C:\Users\kimhyunju\Desktop\PCB_classify\NG_newlabel'의 고유한 이미지 크기:
{(159, 200), (200, 172), (199, 200), (200, 165), (183, 200), (200, 200), (178, 200), (161, 200),
(174, 200), (199, 199), (200, 180), (200, 199), (200, 154), (200, 189), (200, 179), (200, 185)}

서로 다른 크기의 이미지가 있습니다. 고유 크기 목록:

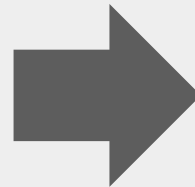
{(159, 200), (200, 172), (199, 200), (200, 165), (183, 200), (200, 200), (178, 200), (161, 200),
(174, 200), (199, 199), (200, 180), (200, 199), (200, 154), (200, 189), (200, 179), (200, 185)}

OK 이미지 크기 확인:

폴더 'C:\Users\kimhyunju\Desktop\PCB_classify\OK'의 고유한 이미지 크기:
{(200, 194), (166, 200), (199, 200), (194, 200), (178, 200), (200, 200), (195, 200), (168, 200),
(161, 200), (199, 199), (200, 193), (200, 199), (200, 169)}

서로 다른 크기의 이미지가 있습니다. 고유 크기 목록:

{(200, 194), (166, 200), (199, 200), (194, 200), (178, 200), (200, 200), (195, 200), (168, 200),
(161, 200), (199, 199), (200, 193), (200, 199), (200, 169)}



NG_newlabel 이미지 크기 변경 중...

이미지 bichim_1.bmp이(가) (224, 224) 크기로 저장되었습니다.

이미지 bichim_2.bmp이(가) (224, 224) 크기로 저장되었습니다.

이미지 bichim_3.bmp이(가) (224, 224) 크기로 저장되었습니다.

이미지 chui_1.bmp이(가) (224, 224) 크기로 저장되었습니다.

이미지 chui_2.bmp이(가) (224, 224) 크기로 저장되었습니다.

이미지 dust_1.bmp이(가) (224, 224) 크기로 저장되었습니다.

이미지 dust_10.bmp이(가) (224, 224) 크기로 저장되었습니다.

이미지 dust_100.bmp이(가) (224, 224) 크기로 저장되었습니다.

...

이미지 ok_992.bmp이(가) (224, 224) 크기로 저장되었습니다.

이미지 ok_994.bmp이(가) (224, 224) 크기로 저장되었습니다.

이미지 ok_995.bmp이(가) (224, 224) 크기로 저장되었습니다.

모든 이미지 크기 변경이 완료되었습니다.

→ ViT와 다양한 모델에서 널리 사용되는 표준 입력 크기이기에,
사전 학습된 모델을 활용할 때 최적의 성능을 얻을 수 있을 것이라 판단

01 EDA 및 데이터 전처리

3. 데이터 전처리

이미지 밝기 확인 및 정규화

NG 이미지 밝기 확인:

폴더 'C:\...\Desktop\PCB_classify\NG_sorted'의 평균 밝기: 127.32

폴더 'C:\...\Desktop\PCB_classify\NG_sorted'의 밝기 표준편차: 35.54

이상 밝기 이미지를 확인합니다.

파일: C:\...\NG_sorted\bichim\bichim_8.bmp, 밝기: 210.67

파일: C:\...\NG_sorted\dust\dust_103.bmp, 밝기: 212.82

파일: C:\...\NG_sorted\dust\dust_104.bmp, 밝기: 205.75

파일: C:\...\NG_sorted\dust\dust_105.bmp, 밝기: 218.08

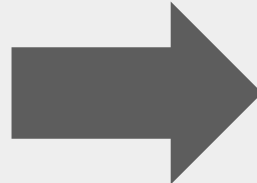
...

short_193.bmp의 밝기를 정규화했습니다.

short_195.bmp의 밝기를 정규화했습니다.

short_202.bmp의 밝기를 정규화했습니다.

모든 이상 밝기 이미지가 정규화되었습니다.

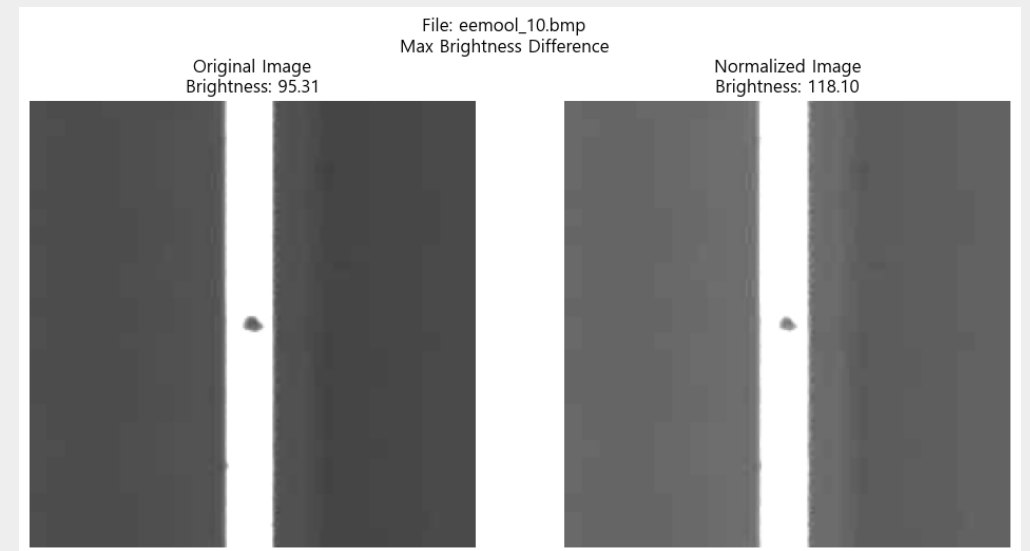


평균 밝기: 127.32 → 123.77
밝기 표준편차: 35.54 → 30.47

NG 이미지 평균 밝기 및 표준편차 계산:

폴더 'C:\...\Desktop\PCB_classify\NG_sorted'의 평균 밝기: 123.77

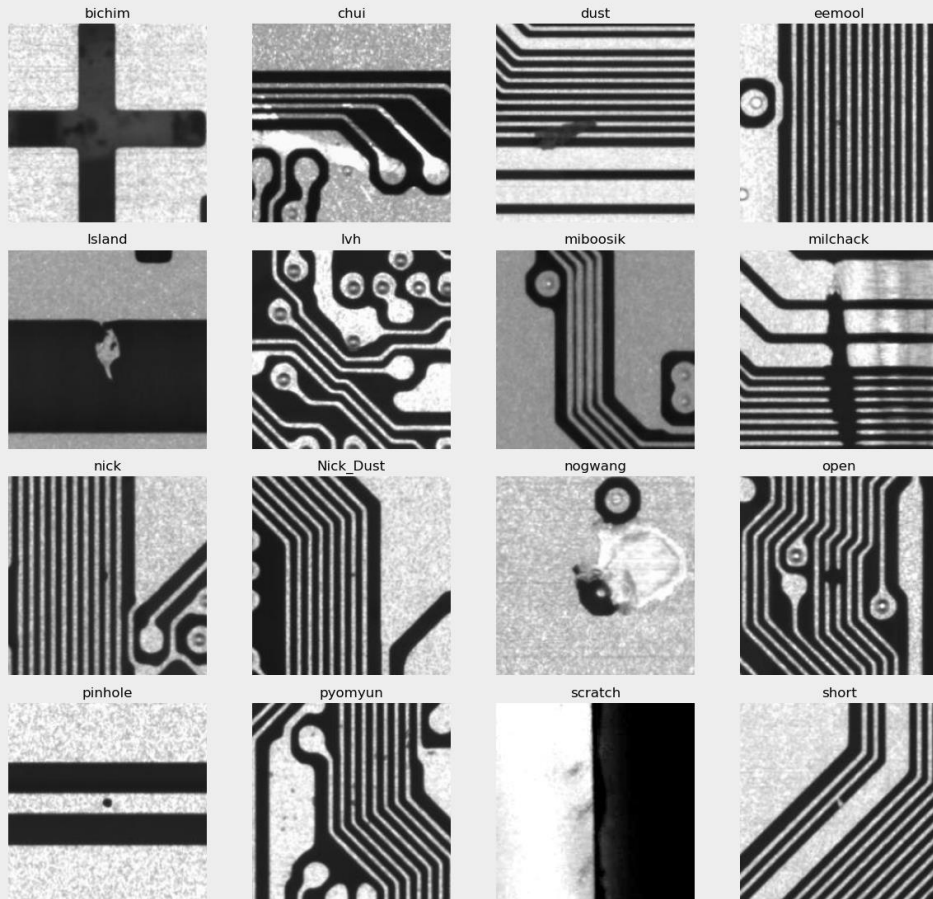
폴더 'C:\...\Desktop\PCB_classify\NG_sorted'의 밝기 표준편차: 30.47



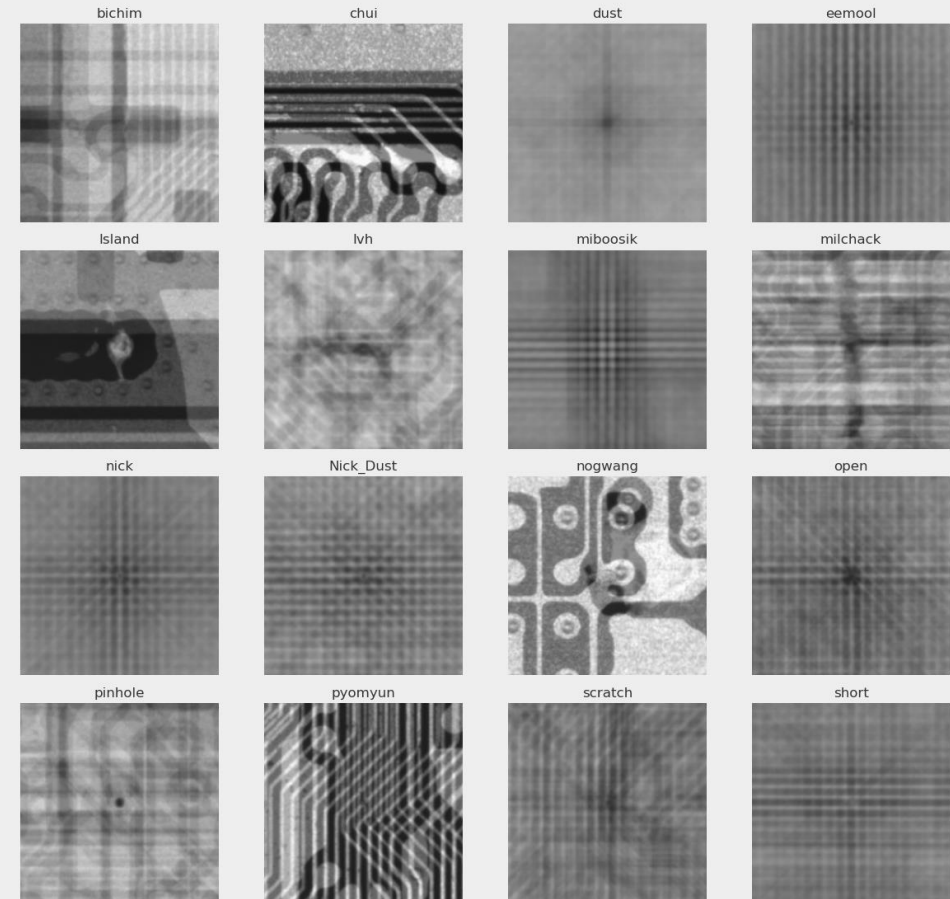
01 EDA 및 데이터 전처리

3. 데이터 전처리

Random Defect Image for Each Label



Average Defect Images for Each Label



02

이진 분류

1. 데이터 라벨링
2. 모델 비교 및 평가
 - 2.1 하이퍼파라미터 설명 - 옵티마이저
3. EfficientNet-B3 모델 평가

02 이진 분류

2. 모델 비교 및 평가

1. CNN

- CNN은 이미지의 공간적 정보를 학습하여 저수준 특징(엣지, 텍스처 등)을 잘 추출함.

2. ResNet

- ResNet은 잔차연결을 사용하여 깊은 네트워크에서도 학습이 잘 이루어지도록 도움.

3. CNN-ResNet

- CNN과 ResNet의 장점을 결합하여 저수준 및 고수준 특징을 모두 효과적으로 학습함.

4. EfficientNet

- EfficientNet은 모델의 크기와 성능 간의 최적화를 위해 폭, 깊이, 해상도를 동시에 조절함.

5. DenseNet

- DenseNet은 각 계층이 이전 모든 계층의 출력을 입력으로 받아, 정보 흐름과 학습을 촉진시킴.

6. MobileNet

- MobileNet은 경량화된 모델로, 깊이 별 분리 합성곱을 통해 모바일 및 임베디드 시스템에서도 사용할 수 있음.

02

이진 분류

2. 모델 비교 및 평가

1. CNN

Batch size: 32
 Learning Rate: 0.001
 Pretrained: False
 Dropout: 0.5
 Early Stopping: 3
 Optimizer: Adam

정확도

Val Set : 0.9042
 Test Set : 0.8750

2. ResNet-18

Batch size: 32
 Learning Rate: 0.001
 Pretrained: True
 Dropout: False
 Early Stopping: 3
 Optimizer: Adam

정확도

Val Set : 0.9489
 Test Set : 0.9131

3. ResNet-50

Batch size: 32
 Learning Rate: 0.001
 Pretrained: True
 Dropout: False
 Early Stopping: 3
 Optimizer: Adam

정확도

Val Set : 0.9170
 Test Set : 0.9195

4. CNN-ResNet50

Batch size: 32
 Learning Rate: 0.001
 Pretrained: True
 Dropout: 0.5
 Early Stopping: 3
 Optimizer: Adam

정확도

Val Set : 0.9170
 Test Set : 0.9153

02 이진 분류

2. 모델 비교 및 평가

5. EfficientNet-B0

Batch size: 32
 Learning Rate: 0.001
 Pretrained: True
 Dropout: False
 Early Stopping: 3
 Optimizer: Adam

정확도

Val Set : 0.9638
 Test Set : 0.9682

6. EfficientNet-B2

Batch size: 32
 Learning Rate: 0.001
 Pretrained: True
 Dropout: False
 Early Stopping: 3
 Optimizer: Adam

정확도

Val Set : 0.9766
 Test Set : 0.9746

7. EfficientNet-B3

Batch size: 32
 Learning Rate: 0.001
 Pretrained: True
 Dropout: False
 Early Stopping: 3
 Optimizer: Adam

정확도

Val Set : 0.9830
 Test Set : 0.9873

8. EfficientNet-B4

Batch size: 32
 Learning Rate: 0.001
 Pretrained: True
 Dropout: False
 Early Stopping: 3
 Optimizer: Adam

정확도

Val Set : 0.9681
 Test Set : 0.9407

02

이진 분류

2. 모델 비교 및 평가

9. DenseNet-121

Batch size: 32
Learning Rate: 0.001
Pretrained: True
Dropout: False
Early Stopping: 3
Optimizer: Adam

정확도

Val Set : 0.8553
Test Set : 0.8602

6. DenseNet-169

Batch size: 32
Learning Rate: 0.001
Pretrained: True
Dropout: False
Early Stopping: 3
Optimizer: Adam

정확도

Val Set : 0.9340
Test Set : 0.9386

7. DenseNet-201

Batch size: 32
Learning Rate: 0.001
Pretrained: True
Dropout: False
Early Stopping: 3
Optimizer: Adam

정확도

Val Set : 0.9489
Test Set : 0.9301

8. MobileNet-V2

Batch size: 32
Learning Rate: 0.001
Pretrained: True
Dropout: False
Early Stopping: 3
Optimizer: Adam

정확도

Val Set : 0.9702
Test Set : 0.9386

02 이진 분류

2.1 하이퍼파라미터 설명 -옵티마이저

1.Adam

장점

- 학습률을 데이터에 맞춰 자동으로 조정하는 적응형 학습률을 사용
- 작은 데이터셋이나 전이 학습 환경에서 좋은 성능

단점

- 과적합을 방지하기 위한 정규화 기법이나 학습률 조정에 민감할 수도 있음

PCB 기판 이미지는 비교적 노이즈가 적고 정확한 패턴을 학습해야 하는 경우가 많아 Adam의 적응형 학습률로 변화에 빠르게 대응할 수 있어 적합하다고 판단함.

2.SGD

장점

- 대규모 데이터셋에서 안정적인 수렴
- 모멘텀을 추가하여 그래디언트를 고려하여 급격한 변화 없이 학습

단점

- 적절한 학습률 튜닝이 필수, 부적절한 학습률 사용할 경우 학습이 매우 느리거나 수렴하지 않을 수 있음
- 적응형 학습률을 사용하지 않기 때문에, 학습률을 수동으로 조정해야 함

PCB 기판 이미지 데이터가 비교적 작은 데이터셋이라고 생각해 SGD가 Adam에 비해 부적절하다고 생각함.

3.RMSProp

장점

- Adam과 유사한 적응형 학습률 사용
- 그래디언트 이동 평균을 추적하여 급격한 변화 없이 안정적인 학습 유도

단점

- Adam보다 보편적으로 사용되지 않아 보임.

Adam이 잘 동작하지 않으면 대안으로 사용해 보려 하였음.

4.AdaGrad

장점

- 학습률을 자동으로 조정하며, 각 파라미터에 대해 학습률을 다르게 적용
- 희소 데이터에 강점

단점

- 시간이 지남에 따라 학습률이 너무 작아져서 유용한 업데이트를 못하게 된다는 문제점이 있음

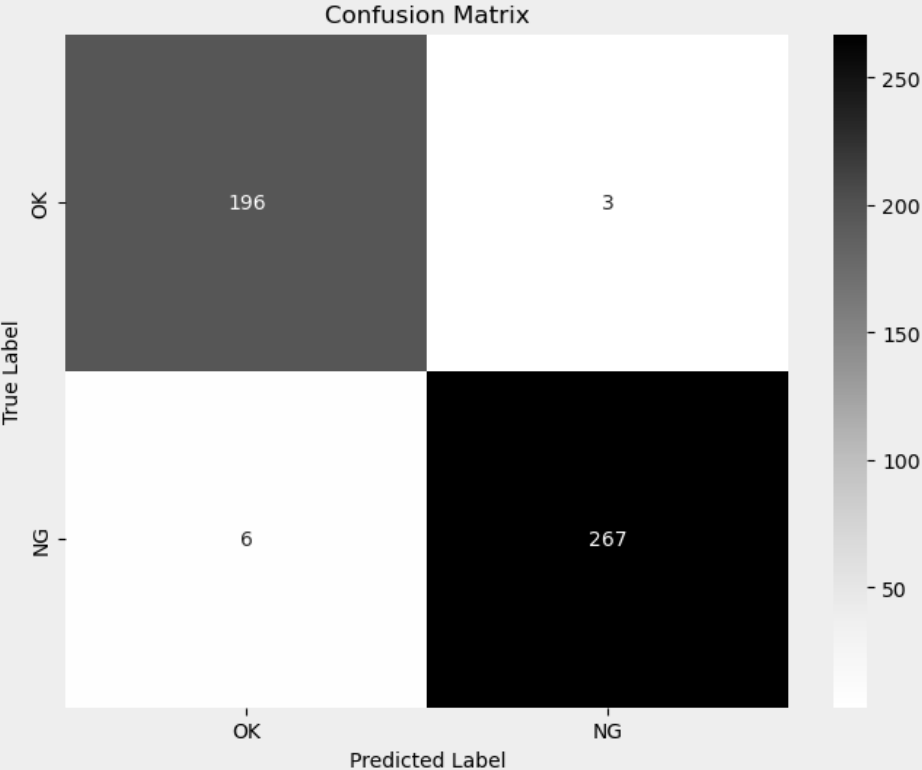
PCB 이미지 특성 상 희소 데이터가 많지 않기 때문에 적합하지 않다고 생각

* 희소 데이터

데이터 내에서 대부분의 요소가 0이거나 비어 있는 경우

02 이진 분류

3. EfficientNet-B3 모델 평가



성능지표

정밀도: 예측 중 실제 정답인 비율
재현율: 정답 중 모델이 맞춘 비율

label	precision	recall	f1-score
OK	0.9703	0.9849	0.9776
NG	0.9889	0.9780	0.9834
정확도			
클래스간 평균 성능	0.9796	0.9815	0.9805
가중치 평균 성능	0.9811	0.9809	0.981

03

다중 클래스 분류

1. 모델 비교
2. 정확도 향상 과정
 - 2.1 증강 후 모델 비교

03 다중 클래스 분류

1. 모델 비교

1. CNN

Batch size: 32
 Learning Rate: 0.001
 Pretrained: False
 Dropout: False
 Early Stopping: 3
 Optimizer: Adam
 Loss Function: Cross Entropy

정확도

Val Set : 0.4710
 Test Set : 0.5290

2. ViT

Batch size: 32
 Learning Rate: 0.001
 Pretrained: True
 Dropout: False
 Early Stopping: 3
 Optimizer: Adam
 Loss Function: Cross Entropy

정확도

Val Set : 0.4203
 Test Set : 0.4819

3. CNN-ViT

Batch size: 32
 Learning Rate: 0.001
 Pretrained: True
 Dropout: False
 Early Stopping: 3
 Optimizer: Adam
 Loss Function: Cross Entropy

정확도

Val Set : 0.5435
 Test Set : 0.5326

4. EfficientNet-B3

Batch size: 32
 Learning Rate: 0.001
 Pretrained: True
 Dropout: False
 Early Stopping: 3
 Optimizer: Adam
 Loss Function: Cross Entropy

정확도

Val Set : 0.6051
 Test Set : 0.5725
 → 정확도가 너무 낮다고 판단

03 다중 클래스 분류

2. 정확도 향상 과정

데이터 증강

증강 방법	예상	결과
TransGAN (참고논문: Conditional TransGAN-Based Data Augmentation for PCB Electronic Component Inspection)	참고한 논문과 다르게 사용한 데이터가 특징이 명확하지 않아, GAN을 이용한 데이터 생성이 잘 되지 않을 것으로 예상함.	예상과 같이 생성된 데이터의 품질이 낮았고 분류 정확도에 기여하지 못함. → GAN을 통한 데이터 증강을 사용하지 않음.
2. 기본 이미지 증강 (회전, 좌우반전)	데이터의 다양한 변형을 통해 모델이 더 많은 패턴을 학습하고 정확도 개선에 도움이 될 것으로 기대함.	데이터 증강을 통해 데이터셋의 다양성을 확보하고 모델이 여러 변형에 대응하도록 함. → 정확도가 눈에 띄게 개선됨.

결함강조

1. 결함 패턴 시각화

2. 피쳐 엔지니어링

3. CNN 기반 특징 추출

4. 클러스터링 기반 패턴 분석

03 다중 클래스 분류

2. 정확도 향상 과정

데이터 증강 - TransGAN

(a) TransGAN의 판별기를 다중 클래스 분류기로 수정하여 원본 데이터셋으로 학습함.

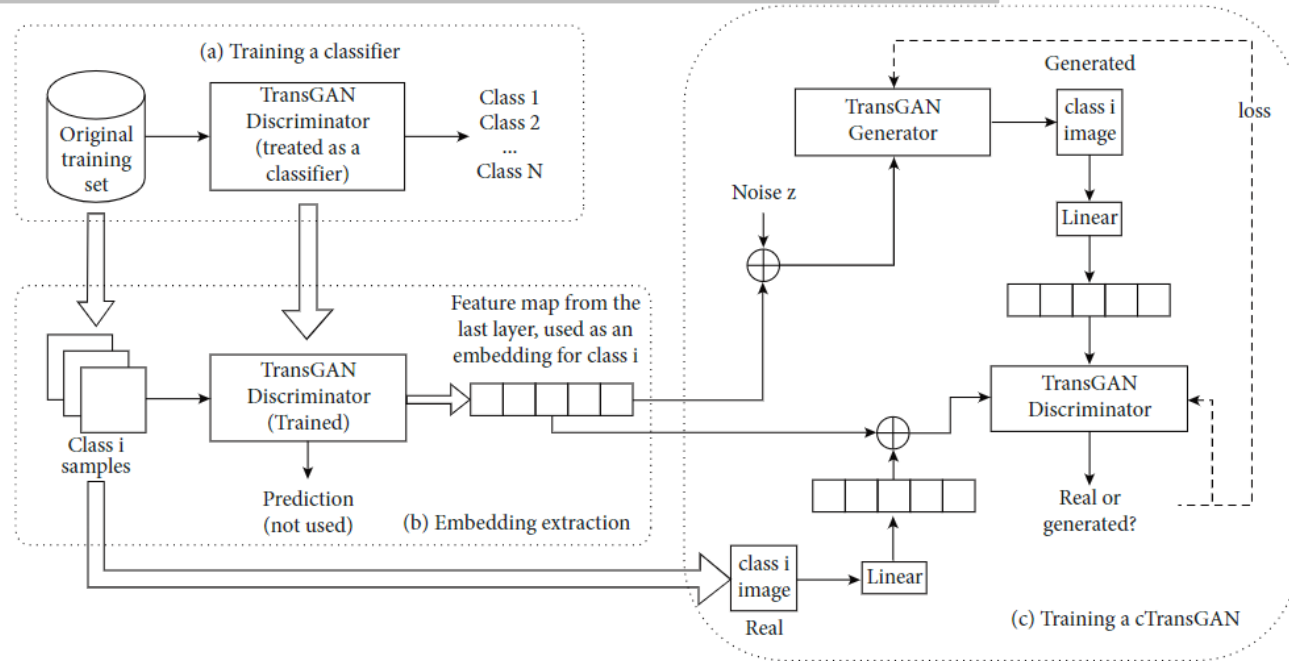
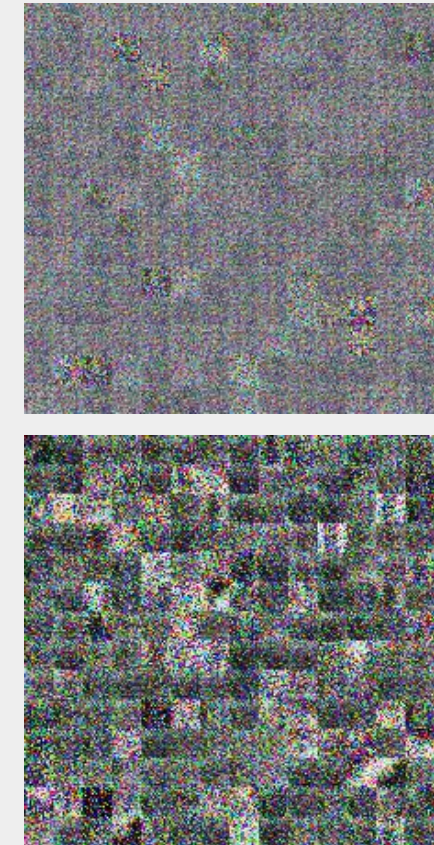


FIGURE 6: The architecture of cTransGAN.

(b) 각 클래스 샘플에 대해 마지막 레이어의 특징을 추출하여 클래스 임베딩(Word2Vec 방식)을 만들.

(c) 생성기가 각 클래스에 맞는 이미지를 생성하도록 유도하여 임베딩과 노이즈 벡터를 결합하여 해당 클래스 이미지를 생성함.



03 다중 클래스 분류

2. 정확도 향상 과정

데이터 증강 - TransGAN

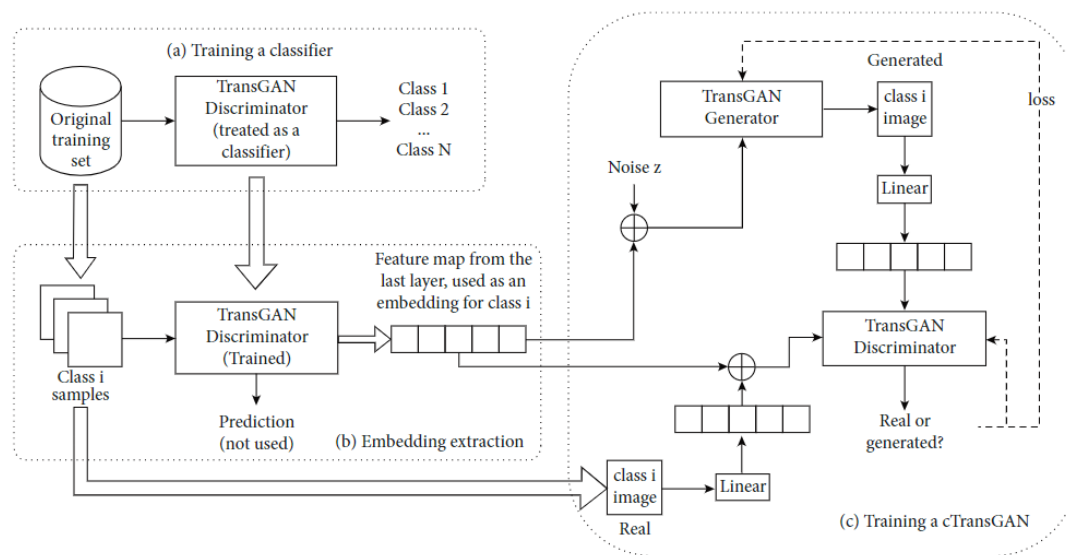


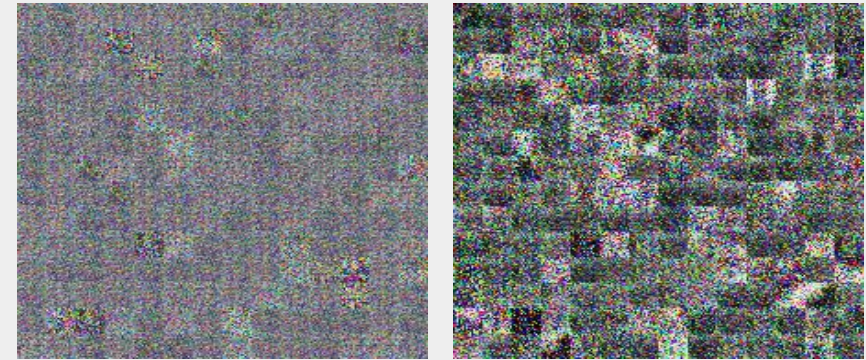
FIGURE 6: The architecture of cTransGAN.

참고논문 내 TransGAN 학습 설정:

Epoch: 240

Batch: 64

Learning Rate: 0.0001(Adam optimizer)



TransGAN 학습 설정:

Epoch: 10

→ 적은 에폭으로 먼저 돌려보고 성능을 확인해보고자 함

→ g_loss (생성자)와 d_loss (판별자)의 손실값이 안정적으로 수렴하지 못하여, 에폭을 크게 잡고 돌려도 효과적이 가짜 이미지 생성이 어려울 것이라고 생각

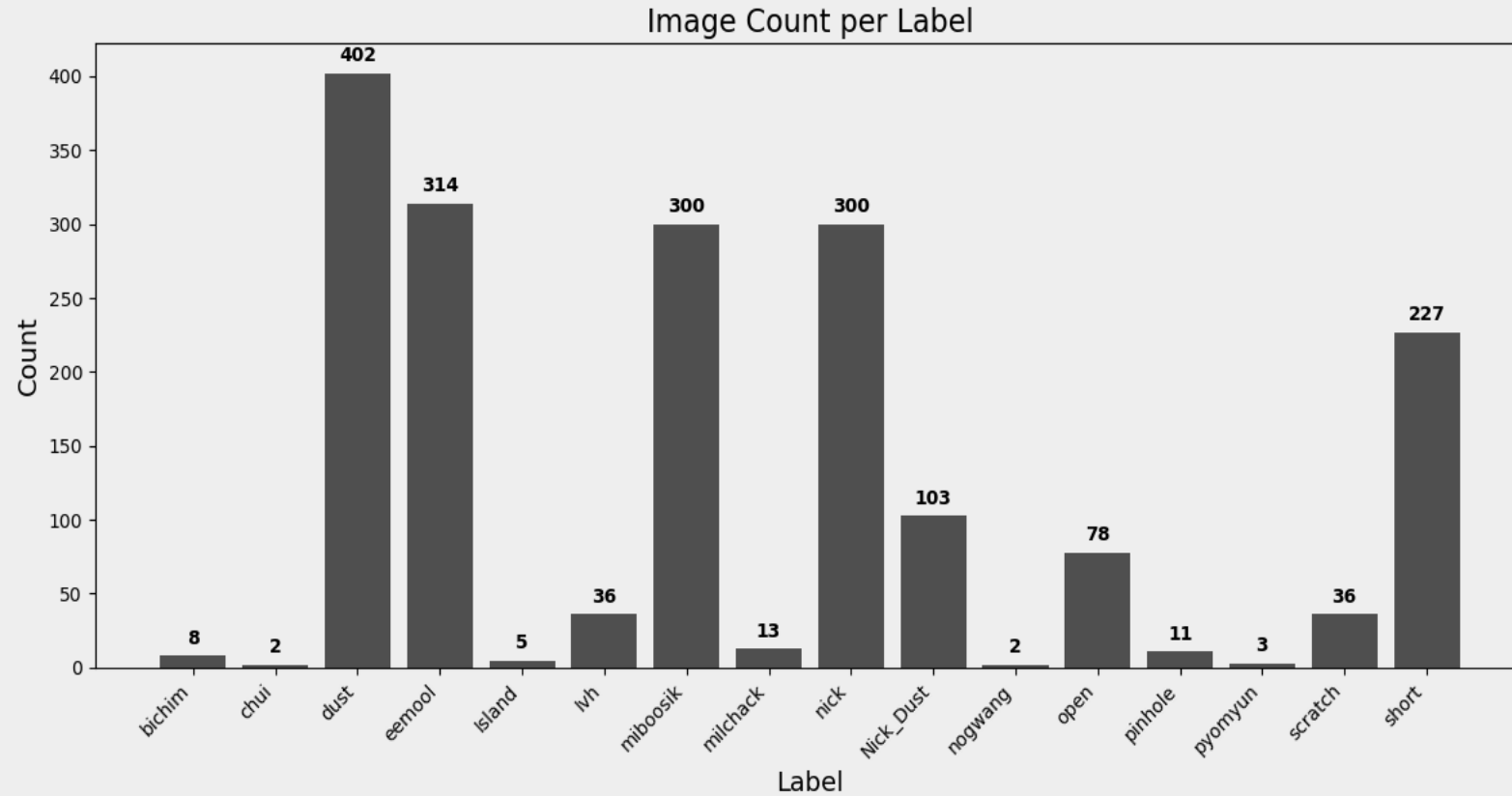
Batch: 32

→ 논문에서 사용한 데이터에 비해 사용하는 PCB 기판의 이미지 데이터가 좀 더 일반화가 어렵다고 생각하여 32로 설정

Learning Rate: 0.0001(Adam optimizer)

03 다중 클래스 분류

2. 정확도 향상 과정



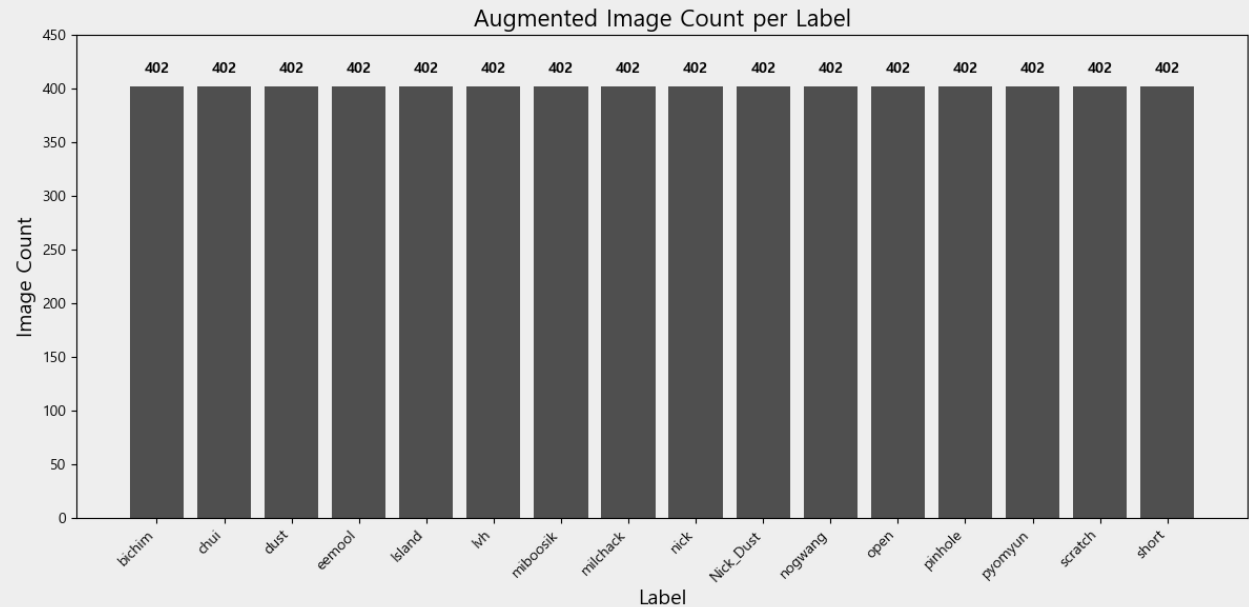
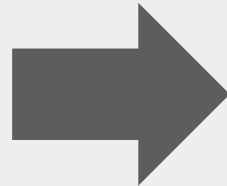
→ 라벨 별 데이터 불균형이 심해 정확도에 영향이 있을 것으로 예상되어
부족한 데이터를 증강하기로 결정

03 다중 클래스 분류

2. 정확도 향상 과정

데이터 증강 - 기본 이미지 증강

bichim 라벨의 이미지 개수: 8
 bichim 라벨의 최종 이미지 개수: 402
 chui 라벨의 이미지 개수: 2
 chui 라벨의 최종 이미지 개수: 402
 dust 라벨의 이미지 개수: 402
 dust 라벨의 최종 이미지 개수: 402
 eemool 라벨의 이미지 개수: 314
 eemool 라벨의 최종 이미지 개수: 402
 Island 라벨의 이미지 개수: 5
 Island 라벨의 최종 이미지 개수: 402
 lvh 라벨의 이미지 개수: 36
 lvh 라벨의 최종 이미지 개수: 402
 ...
 scratch 라벨의 최종 이미지 개수: 402
 short 라벨의 이미지 개수: 227
 short 라벨의 최종 이미지 개수: 402
 데이터 증강 완료.



→ NG 파일의 총 이미지 개수: 4592

03 다중 클래스 분류

2.1 증강 후 모델 비교

1. CNN

Batch size: 32
 Learning Rate: 0.001
 Pretrained: False
 Dropout: False
 Early Stopping: 3
 Optimizer: Adam
 Loss Function: Cross Entropy

정확도

Val Set : 0.8706
 Test Set : 0.8870

2. ViT

Batch size: 32
 Learning Rate: 0.001
 Pretrained: True
 Dropout: False
 Early Stopping: 3
 Optimizer: Adam
 Loss Function: Cross Entropy

정확도

Val Set : 0.9012
 Test Set : 0.9000

3. CNN-ViT

Batch size: 32
 Learning Rate: 0.001
 Pretrained: True
 Dropout: False
 Early Stopping: 3
 Optimizer: Adam
 Loss Function: Cross Entropy

정확도

Val Set : 0.9346
 Test Set : 0.9188

03 다중 클래스 분류

2.1 증강 후 모델 비교

4. EfficientNet-B3

Batch size: 32
 Learning Rate: 0.001
 Pretrained: True
 Dropout: False
 Early Stopping: 3
 Optimizer: Adam
 Loss Function: Cross Entropy

정확도

Val Set : 0.9201
 Test Set : 0.9087

5. EfficientNet-B4

Batch size: 32
 Learning Rate: 0.001
 Pretrained: True
 Dropout: False
 Early Stopping: 3
 Optimizer: Adam
 Loss Function: Cross Entropy

정확도

Val Set : 0.9564
 Test Set : 0.9348

6. EfficientNet-B5

Batch size: 32
 Learning Rate: 0.001
 Pretrained: True
 Dropout: False
 Early Stopping: 3
 Optimizer: Adam
 Loss Function: Cross Entropy

정확도

Val Set : 0.9390
 Test Set : 0.9580

7. EfficientNet-B6

Batch size: 32
 Learning Rate: 0.001
 Pretrained: True
 Dropout: False
 Early Stopping: 3
 Optimizer: Adam
 Loss Function: Cross Entropy

정확도

Val Set : 0.9433
 Test Set : 0.9565

03 다중 클래스 분류

2. 정확도 향상 과정

결함 강조

1. 결함 패턴 시각화

- Grad-CAM으로 특정 결함 부분을 제대로 주목하는지 확인하려 함.
- Attention Mechanism으로 특정 결함 부분을 제대로 주목하는지 확인하려 함.

에러

cuda out of memory
에러로 코랩 환경에서도
시도가 어려웠음.

2. 피쳐 엔지니어링

- 이미지 처리 기술을 적용하여 결함 부위를 강조해서 결함 부분을 더 부각시키는 방법임.
- CLAHE(적응형 히스토그램 평활화)와 같은 기법은 이미지의 대비를 향상시켜 결함 부분을 더욱 두드러지게 만듦.

정확도

증강 전 Best EfficientNet-B3:
0.5725
피쳐 엔지니어링 후: **0.5761** ↑
증강 후 Best EfficientNet-B5:
0.9580
피쳐 엔지니어링 후: **0.9594** ↑

3. CNN 기반 특징 추출

- CNN 기반 특징 추출을 통해 결함의 패턴을 자동으로 추출하여 정확도를 높여보려고 시도함.

정확도

증강 전 Best EfficientNet-B3:
0.5725
특징 추출 후: **0.4891** ↓
증강 후 Best EfficientNet-B5:
0.9580
특징 추출 후: **0.9536** ↓

4. 클러스터링 기반 패턴 분석

- 클러스터링 기반 패턴 분석으로 비지도 학습을 통해 결함 패턴을 분석하여 정확도를 높여보려고 시도함.

정확도

증강 전 Best EfficientNet-B3:
0.5725
패턴 분석 후: **0.5906** ↑
증강 후 Best EfficientNet-B5:
0.9580
패턴 분석 후: **0.9551** ↓

03 다중 클래스 분류

2. 정확도 향상 과정

결함 강조

피쳐 엔지니어링

- 이미지 처리 기술을 적용하여 결함 부위를 강조해서 결함 부분을 더 부각시키는 방법임.

왜 가장 좋은 결과가 나왔을까?

- CLAHE(적응형 히스토그램 평활화)와 같은 기법은 이미지의 대비를 향상시켜 결함 부분을 더욱 두드러지게 만들어서 결함의 엣지나 패턴이 더욱 뚜렷해져 도움이 되었다고 생각함.

CNN 기반 특징 추출

- CNN 기반 특징 추출을 통해 결함의 패턴을 자동으로 추출하여 정확도를 높여보려고 시도함.

왜 아쉬운 결과가 나왔을까?

- CNN 기반 특징 추출은 모델이 자체적으로 이미지에서 중요한 특징을 학습하지만, 원본 이미지의 품질에 크게 의존하기 때문에, PCB의 결함부분이 명확하지 않아 좋은 결과가 나오기 어려웠지 않았나 생각함.

클러스터링 기반 패턴 분석

- 클러스터링 기반 패턴 분석으로 비지도 학습을 통해 결함 패턴을 분석하여 정확도를 높여보려고 시도함.

왜 아쉬운 결과가 나왔을까?

- 클러스터링 기반 패턴 분석은 데이터를 특정 기준에 따라 군집화하여 패턴을 학습하여, 결함이 명확하지 않고 복잡한 패턴이어서 성능이 떨어졌다고 생각함.

END OF DOCUMENT

감사합니다